

SOFTWARE REQUIREMENTS SPECIFICATION

Project Blueprint & Implementation Record

HOTEL MANAGEMENT SYSTEM (HMS)

A Single-Property Desktop Hotel Operations Application

(Electron / Next.js / Node.js-Express / SQLite)

Document Type: SRS + System Design + As-Built Implementation Record

Version: 4.0 (Desktop Stack Revision)

Date: August 2026

This revision supersedes Version 3.0 (Web Stack Revision). The project was ultimately implemented as a single-machine Electron desktop application rather than a hosted web platform. This document has been rewritten to describe the system as actually built and verified, not the earlier cloud-oriented plan.

1. Introduction & Project Overview

1.1 Overview

The Hotel Management System (HMS) is a single-machine desktop application that automates the operational lifecycle of a hotel — room inventory, reservations, check-in/check-out, billing, housekeeping, restaurant service, staff management, and reporting — replacing manual registers and disconnected spreadsheets with one local, shared source of truth.

HMS is delivered as an installable Electron desktop application. On launch, Electron starts a local Node.js/Express API and serves a statically-exported Next.js interface inside its own window, backed by a SQLite database file stored in the operating system's per-user application-data directory. No external server, hosting account, or internet connection is required to run the application day-to-day.

NOTE: The originally-scoped Version 3.0 plan described a networked, multi-device web platform (Next.js on Vercel, Express/PostgreSQL on Railway) so that reception, housekeeping, and restaurant staff could each work from their own device. The system as actually implemented runs as a single desktop application on one machine. This is the most significant architectural difference between that plan and the delivered product, and it is addressed directly throughout this document rather than glossed over.

1.2 Problem Statement

Problem	Impact
Paper-based or spreadsheet booking records	Double bookings, lost or inconsistent records
No live room status	Staff must physically check or ask a colleague whether a room is clean or occupied
Disconnected billing across departments	Manual, error-prone tallying of room, food, and service charges at checkout
No centralized reporting	Management has no consolidated view of occupancy, revenue, or staff activity
Reliance on external/cloud software for a single small property	Recurring hosting costs and an internet dependency that a single front-desk terminal does not need

1.3 Delivered Solution

HMS gives hotel staff one shared, role-restricted desktop application reading from a single local database. Core mechanisms, as actually implemented:

- A room-status engine (Available, Occupied, Reserved, Cleaning, Maintenance) that updates live within the running application via Socket.IO the instant a change is made — instantly reflected across every open screen of the same running instance.
- A booking engine that checks for date-range overlap on the target room inside a database transaction before confirming a reservation, structurally reducing the risk of double-booking. (See Section 4.6 for the specific mechanism and its limits — SQLite does not offer the database-level exclusion constraint a networked PostgreSQL deployment would.)
- An automated billing engine that aggregates room, restaurant, and service charges into one itemized invoice, exportable as a PDF, generated locally.
- A reporting layer with interactive charts (Recharts) giving management visibility into occupancy and revenue without manual compilation.
- A per-account authentication system with username/password login, forced password change on first use, and automatic account lockout after repeated failed attempts.

- An in-app Backup & Restore feature that exports the full database and uploaded files to a single archive, and can restore from one — the desktop equivalent of the managed cloud backups a hosted plan would have used.
- A license-activation layer that binds the installed application to the machine it is installed on (see Section 3.1.16).

NOTE: Google/Microsoft single sign-on, described in the Version 3.0 plan, was not implemented. Login is username-and-password only, matching the actual login screen. There is also no self-registration screen — new accounts are created by an Administrator from within the app, and the login screen's 'Contact your administrator' text reflects this directly.

2. Project Scope & Objectives

2.1 In Scope (Version 1.0 — as delivered)

- Room inventory, room-type categorization, and pricing management, including per-room image upload.
- Full reservation lifecycle: booking (walk-in or phone, staff-entered) → confirmation → check-in → stay → check-out, for both local and foreign guests as two distinct guest-type flows (see FR-032a).
- Role-based accounts for Admin, Receptionist, Housekeeping, and Restaurant staff, created only by an Administrator — there is no public self-registration.
- Room availability checking with an application-level, transaction-guarded overlap check (see Section 4.6).
- Billing, itemized invoicing, and payment recording, with support for partial payments and discounts.
- In-house restaurant / room-service ordering linked to the guest's active booking.
- Housekeeping task tracking per room, created automatically at checkout.
- Staff records and a full audit log of system actions.
- Reports and dashboards: occupancy, revenue, bookings, and length-of-stay, rendered as interactive charts, for a date range chosen by the user.
- Role and permission management, enforced on both the interface and the API — not UI-only.
- In-app, role-targeted, real-time notifications (booking created, check-in, check-out, low balance, etc.).
- In-app database Backup & Restore.
- Light and dark theme support.
- License activation bound to the installed machine.

2.2 Out of Scope (Version 1.0)

- Multi-device / networked access — the application runs on one machine at a time; there is currently no supported way for a second terminal (e.g., a housekeeping tablet) to connect to the same running instance over a network. This is the direct trade-off of choosing a desktop architecture over the Version 3.0 web plan, and is revisited as a Phase 2 candidate in Section 8.5.
- Customer-facing public booking website and native mobile app.
- Third-party OTA integration (Booking.com, Expedia, Agoda).
- Real payment-gateway integration — payments are recorded by staff, not processed online.
- Multi-property / hotel-chain management (single-tenant, single-hotel database).
- Google / Microsoft OAuth or any third-party single sign-on.
- Email-based 'forgot password' flow — there is no outbound email capability in a machine-local desktop app; account recovery is handled by an Administrator instead (see FR-006).

- A fully built-out Guest Feedback module — the underlying database table exists, but no screen or API endpoint currently reads from or writes to it. Treated here as reserved for a future phase, not a current feature (see Section 8.5).

2.3 Objectives

- Eliminate manual, paper-based record-keeping across all departments of a single property.
- Reduce guest check-in/check-out time to under a few minutes.
- Minimize double-bookings through a transaction-guarded, application-level availability check.
- Give front-desk, housekeeping, and restaurant staff a shared live view of room status within the running application.
- Provide management with on-demand reports on occupancy and revenue without manual compilation.
- Deliver an interface simple enough that new front-desk staff are productive after minimal training.
- Require no ongoing hosting cost, external database service, or internet connection for day-to-day operation.

3. Requirements Analysis

Requirement IDs are preserved from the Version 3.0 document where the underlying requirement still applies, so this document can be diffed against the original plan. Requirements that no longer apply to the delivered architecture are explicitly marked as removed, with the reason given, rather than silently dropped.

3.1.1 Authentication & Access Control

ID	Requirement	Status
FR-001	The system shall allow an Administrator to create a staff account with a username, name, phone, and role. There is no self-registration screen.	Revised — was open self-registration by email
FR-002	The system shall allow login via username and password, with credentials verified against a bcrypt-hashed password.	Revised — username, not email
FR-003	Google / Microsoft OAuth 2.0 login.	Removed — not implemented
FR-004	The system shall issue a signed JWT access token and refresh token on successful login, scoped to the user's role, with silent refresh on expiry so an active session is not interrupted.	As delivered
FR-005	The system shall lock an account for a cooldown period after 5 consecutive failed login attempts, showing the user how many attempts remain before lockout.	As delivered
FR-006	A 'Forgot Password' flow that emails a reset link.	Removed — no outbound email capability; login screen instead directs the user to contact their Administrator
FR-007	New accounts (and legacy accounts restored from an older backup) are created with a randomly generated temporary password and a forced password-change requirement on first login — the account cannot be used until a new password is set.	New — added during hardening

3.1.2 Admin Management

ID	Requirement
FR-010	Admin can create, update, and deactivate staff accounts; role assignment is validated against existing roles.
FR-011	Admin can create, rename, and delete roles, and assign granular permissions per role. Deleting a role in use by an existing account is blocked; renaming a role updates every account that references it.
FR-012	Admin can configure hotel-wide settings across dedicated General, Hotel Profile, Tax, and Currency sections, including hotel name, logo, address, tax rate, and currency symbol.
FR-013	Admin can view a full, append-only audit log of system actions, filterable by user, module, and date. Log entries cannot be edited or deleted from the interface.

3.1.3 Room Management

ID	Requirement
FR-020	The system shall support configurable room types with distinct pricing.
FR-021	Each room record stores: number, floor, type, price, status, amenities, and an optional uploaded photo.
FR-022	Room status changes are broadcast live via Socket.IO to every open screen within the same running application instance.
FR-023	A room cannot be deleted while it has an active (Confirmed or Checked-In) booking; deleting a room with only historical bookings is handled without breaking that historical data.

3.1.4 Booking & Reservation Management

ID	Requirement
FR-030	Staff can search rooms by date range and room type, and the system returns only rooms with no confirmed or checked-in booking overlapping that range.
FR-031	The system shall display matching available rooms with price and photo.
FR-032	Receptionist can create a booking for a walk-in or phone guest.
FR-032a	The system shall support two distinct guest-type booking flows — Local and Foreign — with different default identification fields (CNIC vs. Passport) and nationality handling, sharing one underlying booking form driven by the selected guest type.
FR-033	The system shall calculate nights, subtotal, tax, and total automatically as the booking is built, using precise decimal arithmetic rather than floating-point math, to avoid rounding drift in billing.
FR-034	The system shall allow booking modification and cancellation, subject to role permissions. A cancelled booking is excluded from occupancy and check-in/out counts elsewhere in the system.

3.1.5 Room Availability

ID	Requirement
FR-040	The system shall check availability against check-in date, check-out date, and room, at the moment a booking is created.
FR-041	The system shall reduce the risk of double-booking by performing the overlap check and the room-reservation write inside a single database transaction, rather than as two separate, unguarded steps. This is an application-level safeguard, not a database-level exclusion constraint — SQLite, used for local single-machine storage, has no equivalent to a PostgreSQL exclusion constraint. See Section 4.6 for a

ID	Requirement
	direct discussion of this trade-off.

3.1.6 Check-In System

ID	Requirement
FR-050	Receptionist can search a booking by guest name, room number, or booking reference.
FR-051	The system shall display full booking details and the assigned room before check-in is confirmed.
FR-052	On check-in, the system shall, in one atomic transaction, set booking status to Checked-In and room status to Occupied, and push both updates live to every open screen.

3.1.7 Check-Out & Room Allocation

ID	Requirement
FR-060	Receptionist can initiate checkout from the active-guest list.
FR-061	The system shall compile all charges — room nights, restaurant orders — into a single folio/invoice, generated once and updated in place (not recreated) if further charges or payments are added, so the invoice keeps a stable identity throughout the guest's stay.
FR-062	On checkout, the system shall create a housekeeping task for the room. Per the current configuration, the room is made immediately available for a new booking at the moment of checkout, rather than waiting for that housekeeping task to be marked complete.

NOTE: FR-062 is stated as an explicit, deliberate configuration choice, confirmed directly during the project's hardening review, not an oversight. An earlier implementation gated room availability on housekeeping-task completion; this was intentionally changed. Anyone maintaining this system should treat the current 'immediately available' behavior as the intended design unless a future decision changes it again — it is called out here specifically so it isn't mistaken for a defect.

3.1.8 Payment & Billing

ID	Requirement
FR-070	The system shall support a staff-selectable payment method per transaction (e.g., Cash, Card).
FR-071	The system shall record partial payments and track the remaining balance; a payment that would exceed the remaining balance is rejected rather than silently accepted.
FR-072	The system shall support applying a discount to an invoice.

3.1.9 Invoice Generation

ID	Requirement
FR-080	The system shall generate an invoice as charges and payments are recorded against a booking.
FR-081	The invoice shall include hotel details (pulled from Hotel Settings), guest details, itemized charges, tax, and total.
FR-082	The system shall render the invoice as a downloadable PDF, generated locally on the same machine — no external rendering service is used.

3.1.10 Staff Management

ID	Requirement
FR-100	The system shall store staff records: name, role, department, contact, and status.
FR-101	New accounts are issued a randomly generated per-account temporary password (never a shared default) and must set their own password before first use, per FR-007.

3.1.11 Food / Restaurant Services

ID	Requirement
FR-110	Restaurant staff can view current in-house guests.
FR-111	Staff can create a food/room-service order against a room, with items and quantities, restricted to rooms with an active (Checked-In) booking so charges cannot be attached to an unoccupied room.
FR-112	New and updated orders appear live on the Restaurant order queue via Socket.IO within the running application.

3.1.12 Housekeeping Management

ID	Requirement
FR-120	The system shall automatically create a housekeeping task once checkout is completed.
FR-121	Housekeeping tasks can be assigned and tracked per staff member.
FR-122	Staff can mark a task complete; the system correctly accounts for a room having more than one open task at once before considering it fully clean. (Per FR-062, task completion is no longer what gates the room's booking availability — see the note under Section 3.1.7.)

3.1.13 Reports & Statistics

ID	Requirement
FR-130	The dashboard shall show today's occupancy, revenue, check-ins, and check-outs as summary cards, each with a real trend comparison against the prior day's actual figures — not a placeholder value.
FR-131	The system shall produce occupancy, revenue, and booking-volume reports over a user-selected date range, rendered as interactive charts, correctly scoped to that range.
FR-131a	Where a trend metric cannot be honestly computed from data the system currently retains (for example, a historical occupancy-rate comparison, since room status is stored as a live/current value rather than a timestamped history), the system shall return that metric as explicitly unavailable rather than fabricate a plausible-looking number.

3.1.14 Feedback & Review System

ID	Requirement	Status
FR-140 / FR-141	Guest post-stay feedback with multi-category ratings, and manager notification on negative feedback.	Reserved — database table exists; no screen or API implements it yet. Treated as a Phase 2 item (Section 8.5), not a current feature.

3.1.15 Notifications

ID	Requirement
FR-150	The system shall provide an in-app notification bell with an unread count, updated live via Socket.IO, correctly scoped so a user only ever sees their own notifications.
FR-151	Trigger events include: new booking, check-in, check-out, and food order, targeted to the relevant role(s).

3.1.16 Licensing & Activation

ID	Requirement
FR-160	The application shall validate a license record against the machine it is installed on (hardware-derived identifier) before allowing continued use, tracking activation date and expiry.
FR-161	The system shall tolerate minor, expected changes to the machine's hardware-derived identifier (for example, caused by a network adapter going offline) without incorrectly invalidating an otherwise-valid license.

NOTE: This module was not part of the Version 3.0 plan and was found during the final documentation pass, not previously reviewed as part of this project's security hardening work. The fallback encryption key used when no environment-provided license secret is configured is a fixed value committed directly in source. This is the same category of issue already identified and fixed once before in this project (a shared, publicly-committed secret) and is flagged here as something worth a dedicated review, separately from this document.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	The main dashboard shall load within 2 seconds on the machine the application is installed on.
NFR-002	Response Time	A room-availability check shall complete in under 500 milliseconds against the local database.
NFR-003	Real-Time	Room-status and notification updates shall reach every open screen of the running application within 1 second, via the local Socket.IO channel.
NFR-004	Scalability	Revised: the system is explicitly single-instance and single-writer by design — one Electron process per installation, backed by one local SQLite file. The stateless-backend / multi-instance / load-balancer requirement from Version 3.0 does not apply to this architecture and has been removed rather than left as an unmet aspiration.
NFR-005	Security	Revised: no HTTPS/WSS requirement applies, since the application does not communicate over a network by default. Each installation generates its own random JWT signing secrets on first run, stored locally — no secret is shared across installations. Passwords are bcrypt-hashed. See Section 3.1.16's note regarding the license-encryption secret as an open exception to this pattern.
NFR-006	Availability	Revised: no hosted-uptime target applies. Data durability instead depends on the in-app Backup & Restore feature (Section 3.1.16 in the original numbering is Licensing; Backup & Restore is documented under Admin Management, FR-012 area) being used regularly, since there is no automatic managed backup.
NFR-007	Platform Support	Revised: this is a desktop application, not a browser application. The current implementation targets Windows specifically (its local-storage bootstrap logic uses the Windows AppData convention); running on macOS or Linux would require additional path-handling work before it could be verified.

ID	Category	Requirement
NFR-008	Responsiveness	Revised: the interface is designed for a single desktop window rather than a range of device viewports; the tablet/phone-browser responsiveness target from Version 3.0 does not apply.
NFR-009	Auditability	Every create/update/delete on a booking, payment, or room status shall be recorded in the audit log with actor, timestamp, and action — verified as append-only from the interface.
NFR-010	Data Integrity	Money values shall be calculated using precise decimal arithmetic throughout booking, billing, and restaurant charges — not floating-point math — to avoid rounding drift in guest invoices.

3.3 User Roles & Permissions

Permissions are enforced on both the interface (buttons hidden/disabled appropriately) and independently at the API level, so a restricted action is rejected server-side even if attempted directly, not just hidden from view.

Feature	Admin	Receptionist	Housekeeping	Restaurant
System settings	Full	—	—	—
Staff & role management	Full	—	—	—
Room management	Full	View	—	—
Bookings & check-in/out	Full	Full	—	—
Billing & payments	Full	Full	—	—
Housekeeping tasks	Full	View	Full (own tasks)	—
Restaurant orders	Full	View	—	Full
Reports & analytics	Full	View	—	—
Audit logs	Full	—	—	—

NOTE: Exact role names and the full permission matrix are configurable by an Administrator via the Roles/Permissions screens (FR-011); the table above reflects the default roles seeded with a new installation.

3.4 System Constraints

- Version 1.0 targets a single hotel property, running on a single machine.
- The application requires no internet connection for normal day-to-day operation — this is a strength of the desktop architecture relative to the original web plan, not a limitation.
- Payments are recorded by staff, not processed through a live payment gateway.
- A single local SQLite database file is used; it is not designed for more than one application process writing to it at the same time.
- Multi-terminal simultaneous access (e.g., a separate housekeeping tablet on the hotel's network) is not currently supported and would require networked-deployment work not in this version's scope.

3.5 Assumptions

- The hotel has a fixed, known room inventory organized into a small number of room types.
- Staff install the desktop application on the machine(s) they use it from; no browser or separate client software beyond the installer is required.

- Tax rate and currency are configurable per hotel, not hardcoded, and are cached briefly in memory for performance with an explicit invalidation on save.
- The property has one primary working terminal, or is comfortable with the current single-instance constraint until a networked version is built.

4. System Architecture & Technology Stack

4.1 Architectural Decision

Two directions were evaluated at the start of this project: a networked 3-tier web application (as described in Version 3.0 of this document) and a self-contained desktop application with a local, embedded database. The project was ultimately built and shipped as the desktop option.

This was a deliberate trade-off, not an oversight: a desktop build removes recurring hosting costs, removes the internet-connectivity requirement for a small property's day-to-day operation, and simplifies deployment to "run an installer." The cost of that trade-off is real and is stated plainly rather than hidden: true multi-device, multi-department live access — the core promise of Section 1.3 in the Version 3.0 plan — is not currently available, because everything runs inside one Electron process on one machine. If a hotel genuinely needs reception, housekeeping, and the restaurant to work from separate physical terminals at the same time, the current architecture does not deliver that, and the Version 3.0 web approach (or a networked evolution of this desktop app, see Section 8.5) would be the correct direction instead.

4.2 Technology Stack (as implemented)

Layer	Technology	Notes
Desktop shell	Electron	Spawns the compiled backend and serves the statically-exported frontend from one local port; bootstraps the SQLite database, uploads folder, and JWT secrets into the OS AppData directory on first run
Frontend	Next.js 14 (App Router), TypeScript	Built with `output: 'export'` — a fully static build, since there is no Next.js server running at runtime, only the Express server serving the exported files
Styling / UI	Tailwind CSS, lucide-react icons	—
Charts	Recharts	Revenue, occupancy, and bookings charts on the Dashboard and Reports screens
Client data fetching	A centralized fetch client (`lib/api.ts`) with automatic token-refresh handling; TanStack Query used for caching on the Dashboard	—
Real-Time Layer	Socket.io (client + server)	Pushes room-status, booking, order, and notification events to every open screen within the same running instance
Backend API	Node.js + Express	Hosts the REST API and the Socket.io server on one port, alongside the static frontend files
ORM	Prisma	Type-safe queries against the schema in Section 6; migrations tracked in source control
Database	SQLite	A single local file, relocated from the read-only install directory to the OS's per-user AppData directory on

Layer	Technology	Notes
		first run so it remains writable after packaging
Authentication	JWT (access + refresh tokens) with silent refresh, bcrypt password hashing	No OAuth. Each installation generates its own random JWT secrets on first run rather than shipping one shared secret
PDF Generation	PDFKit	Server-side, fully local invoice rendering — no external rendering service
Caching	In-process TTL cache for frequently-read settings (e.g., tax configuration), invalidated explicitly on save	Replaces the Redis layer proposed in Version 3.0, which requires a separate server process not appropriate for a single-machine desktop app
File Storage	Local filesystem, under the AppData uploads directory	Replaces the cloud object storage (Cloudinary/S3) proposed in Version 3.0
Packaging & Distribution	electron-builder	Produces a standalone installer; there is no separate hosting step for frontend or backend

4.3 API Structure

The frontend and backend remain separated by a versioned REST/WebSocket API even though both run on the same machine — the API layer was not collapsed away when the project moved from the web plan to the desktop plan. This means a future networked or multi-device evolution (Section 8.5) is an additive change to how the API is exposed, not a rewrite of how it works.

Resource	Representative Endpoints
Auth	POST /api/auth/login, POST /api/auth/refresh
Rooms	GET /api/rooms, GET /api/rooms/availability, PATCH /api/rooms/:id
Bookings	POST /api/bookings, GET /api/bookings/:id, PATCH /api/bookings/:id, POST /api/bookings/:id/checkin, POST /api/bookings/:id/checkout
Payments / Billing	GET /api/bookings/:id/folio, POST /api/payments
Invoices	GET /api/invoices/:id/pdf
Housekeeping	GET /api/housekeeping/tasks, PATCH /api/housekeeping/tasks/:id
Restaurant	POST /api/restaurant/orders, GET /api/restaurant/menu
Staff / Roles	CRUD under /api/staff and /api/roles, permission-checked per role
Reports	GET /api/reports/summary and related date-scoped report endpoints
Settings	GET/PATCH /api/settings/*, including /api/settings/backup and /api/settings/restore
Notifications	GET /api/notifications, PATCH /api/notifications/:id/read
Audit Logs	GET /api/audit-logs (read-only)
License	GET /api/license/status
Realtime (Socket.io events)	room:status_changed, booking:created, order:created, notification:new

4.4 Deployment Strategy (as implemented)

- The frontend is built via `next build` (static export) and the backend via `tsc`; both are bundled into the Electron package by electron-builder as local resources — there is no separate hosting step for either.

- There is no continuous-deployment pipeline; builds are produced and packaged on demand rather than triggered by every push to the main branch.
- Prisma migrations are applied to the shipped starter database ahead of packaging, and the resulting file is bundled as the installation's initial data.
- On first run, the Electron main process copies the bundled database, an uploads folder, and a freshly-generated `.env` (containing randomly generated JWT secrets) into the OS AppData directory, since the installation directory itself is read-only after packaging.`
- Backups are user-triggered from within the application (Section 3.1.2 / FR-012 area) rather than an automatic managed cloud backup — there is currently no scheduled/automatic backup.

5. UI/UX Design

5.1 Design Principles

- Clarity over decoration — front-desk staff must find information in seconds.
- Consistent color coding for room status: green = Available, red = Occupied, blue = Reserved, yellow = Cleaning, grey = Maintenance.
- A collapsible sidebar drives navigation, matching the layout convention of professional PMS/SaaS dashboards.
- Both light and dark themes are supported and persist across sessions; dark is the default.
- The interface is built for a single desktop window rather than a range of device sizes, though core screens remain usable at smaller widths.

5.2 Screen Flow

Login Screen (username/password only)

- → Dashboard → summary cards, live room-status grid, recent bookings, revenue/occupancy charts, notifications
- → Room Management, Booking, Guests (Local/Foreign), Check-In, Check-Out, Billing
- → Housekeeping → Task List, Mark Complete
- → Restaurant → Menu Management, New Order, Order Queue
- → Staff, Roles & Permissions, Notifications, Reports & Analytics
- → Settings → Account, Security, General, Hotel Profile, Tax, Currency, Roles, Permissions, Backup & Restore, Audit Logs

5.3 Core Screens (as implemented)

Login Screen

A single centered card: hotel branding (name/logo, pulled live from Hotel Settings), a Username field, a Password field with a show/hide toggle, a 'Remember me' checkbox that retains the username (not the password) for next time, and a 'Contact your administrator' line in place of a self-registration link. There is no OAuth button — that was part of the earlier web-plan design, not the delivered product. A forced 'Set New Password' screen replaces the normal form the first time an account with a temporary or restored password logs in.

Main Dashboard

Top summary cards: Today's Check-ins, Today's Check-outs, Occupancy Rate, Today's Revenue, Available Rooms, Reserved Rooms — each with a real, calculated delta against the previous day's actual figures. Below that: a Recent

Bookings table, a color-coded live Room Status grid, a Revenue Trend chart, an Occupancy chart, and a live Notifications feed.

Booking Screen

Guest details (with a Local/Foreign toggle changing the default ID-type field between CNIC and Passport), check-in/check-out date selection with an automatically-calculated night count, a live-filtered list of currently available rooms for the selected dates, and a running total (nights × rate, plus tax) before confirming.

Check-Out / Billing Screen

An itemized folio (room charges, restaurant charges, tax, any discount), the running balance, a payment-method selector, and a combined 'Process Payment' action that — once the balance is fully paid — atomically completes checkout in the same transaction as recording the payment.

6. Database Design

6.1 Entity-Relationship Overview

The database is built around a central Booking table linking a Guest to a Room for a date range. Payments, invoices, and food orders trace back to a booking; housekeeping and maintenance records trace back to a room; Users (staff) are kept separate from Guests, since staff authenticate into the system while guests are the subjects being managed. All primary keys are UUID strings, generated by Prisma, and the schema is defined and versioned through Prisma migrations rather than hand-written SQL — the SQLite file distributed with a fresh installation is generated from this same schema.

6.2 Key Relationships Summary

- Role (1) → User (many); User (1) → Staff (1, optional).
- Guest exists independently of User, since guests do not log in.
- RoomType (1) → Room (many); Room (1) → Booking (many, overlap reduced by an application-level transactional check — see Section 4.6/FR-041, not a database-level exclusion constraint).
- Guest (1) → Booking (many); Booking (1) → Payment (many), (1) → Invoice (1, stable identity — updated in place rather than recreated as charges/payments change), (1) → FoodOrder (many).
- Invoice (1) → InvoiceItem (many); FoodOrder (1) → OrderItem (many).
- Room (1) → HousekeepingTask (many) and (1) → RoomMaintenance (many).
- User (1) → Notification (many) and (1) → AuditLog (many).
- A Feedback table exists in the schema (see FR-140/141) but has no corresponding API or screen in the current version.
- A License table stores a single machine-bound activation record (see FR-160/161).

6.3 A note on the availability guarantee (FR-041)

The Version 3.0 plan specified a PostgreSQL exclusion constraint as the mechanism preventing double-booking — a true database-level guarantee that holds even if application code has a bug. SQLite has no equivalent feature. The delivered system instead performs the overlap check and the room-reservation write inside one database transaction, which meaningfully reduces the risk but is not the same category of guarantee as a database constraint. This is stated plainly here because it is the single most important technical trade-off in the move from the Version 3.0 architecture to the delivered one, and it should be revisited if this project is ever migrated to a networked, multi-writer deployment (Section 8.5).

8. Development Record

Unlike a forward-looking project plan, this section documents how the delivered system actually came together, since the application already exists and has been through multiple rounds of review.

8.1 Development Phases (as completed)

Phase	Deliverables
1. Foundation	Electron/Next.js/Express/Prisma project structure, SQLite schema and seed data, username/password authentication, role-based routing.
2. Core Operations	Room CRUD, guest management (Local/Foreign), booking, availability checking, check-in, check-out.
3. Billing & Real-Time	Billing engine, PDF invoice generation, Socket.io integration for live room-status and notifications, housekeeping and restaurant modules.
4. Reporting & Polish	Reports & analytics dashboards, staff/role/permission management, audit log, in-app backup & restore, light/dark theming.
5. Desktop Packaging & Hardening	Electron packaging via electron-builder, AppData bootstrapping for the database/uploads/secrets, and an extended round of security and data-integrity hardening: per-install random JWT secrets, forced password change on new/restored accounts, transaction-guarded booking overlap checks, decimal-safe billing math, stable invoice identity across payments, server-side permission enforcement, and repo/build hygiene fixes.

NOTE: Phase 5 in particular was iterative: successive review rounds identified and closed a series of real issues — including a shared default staff password, a booking double-write race condition, an invoice identity bug where the invoice's own ID changed on every payment, a backup-restore file leak, and a data-drift issue where the shipped database's admin password silently diverged from its own documentation. Each is described here in outcome only; this document is not the place for the full defect log.

8.2 Testing Strategy (as implemented)

Level	Focus
Unit / Service Tests	Vitest tests over booking and billing service functions, including the overlap-check and checkout logic, run via `npm test` in the backend.
Integration Tests	A Prisma-backed integration test exercising booking creation through to invoice line items.
Manual Functional Testing	A full manual pass across every module (Dashboard through Logout, including every Settings sub-tab), covering functional, UI, validation, security, and edge-case behavior.

Critical test scenarios verified: concurrent/rapid double-booking attempts on the same room and dates; checkout with a zero balance and with a partial payment; correct room-status transitions across the guest lifecycle; account lockout after 5 failed logins; and invoice-total accuracy across room, food, and tax charges.

8.3 Known Limitations & Open Items

- Single-machine, single-writer architecture — no supported multi-terminal deployment yet (Section 4.1).
- Windows-specific local storage bootstrapping — other platforms not currently verified (NFR-007).
- No automatic/scheduled backup — Backup & Restore is user-triggered.
- The Feedback module exists only as an unused database table.

- The license-activation subsystem's fallback encryption secret is a fixed value in source rather than a per-install generated one, unlike the JWT-secret pattern used elsewhere in the app — flagged for a follow-up review, not yet remediated.

8.4 Future Improvements (Phase 2 Candidates)

- Expose the existing REST/WebSocket API over the local network so a genuine second terminal (e.g., a housekeeping tablet) can connect to one running instance — the most direct way to close the multi-device gap versus the original Version 3.0 vision, without a full rewrite to a hosted web architecture.
- Build out the Feedback module end-to-end (screen + API) against the schema that already exists for it.
- Replace the license subsystem's fixed fallback secret with a per-install generated value, consistent with how JWT secrets are already handled.
- Verify and, if needed, adapt the AppData bootstrapping logic for macOS/Linux.
- Integration with a real payment gateway.
- Multi-property management, should this ever need to run more than one hotel.

9. Summary

This revision replaces the Version 3.0 web-architecture plan with a description of the system as it was actually built and subsequently hardened: an Electron desktop application wrapping a local Node.js/Express + Prisma/SQLite backend and a statically-exported Next.js frontend, packaged with electron-builder and requiring no external hosting or internet connection for daily use. The functional requirements, database relationships, and role/permission model largely carry over from the original plan since they describe hotel operations rather than implementation technology — the changes that matter are concentrated in authentication (username-based, no OAuth), availability guarantees (application-level rather than database-level), real-time scope (one running instance rather than many networked devices), and deployment (a packaged installer rather than a hosted service). Where the delivered system differs from the original plan in a way that affects a real guarantee — booking-overlap protection and the room-availability-after-checkout behavior chief among them — this document says so directly rather than describing the aspirational version.